

Revisiting Graph-Tokenizing Large Language Models: A Systematic Evaluation of Graph Token Understanding

Zhongjian Zhang^{*1} Yue Yu^{*1} Mengmei Zhang² Junping Du¹ Xiao Wang^{†3} Chuan Shi^{†1}

Abstract

The remarkable success of large language models (LLMs) has motivated researchers to adapt them as universal predictors for various graph tasks. As a widely recognized paradigm, Graph-Tokenizing LLMs (GTokenLLMs) compress complex graph data into graph tokens and treat them as prefix tokens for querying LLMs, leading many to believe that LLMs can understand graphs more effectively and efficiently. In this paper, we challenge this belief: *Do GTokenLLMs fully understand graph tokens in the natural-language embedding space?* Motivated by this question, we formalize a unified framework for GTokenLLMs and propose an evaluation pipeline, **GTEval**, to assess graph-token understanding via instruction transformations at the format and content levels. We conduct extensive experiments on 6 representative GTokenLLMs with GTEval. The primary findings are as follows: (1) Existing GTokenLLMs do not fully understand graph tokens. They exhibit over-sensitivity or over-insensitivity to instruction changes, and rely heavily on text for reasoning; (2) Although graph tokens preserve task-relevant graph information and receive attention across LLM layers, their utilization varies across models and instruction variants; (3) Additional instruction tuning can improve performance on the original and seen instructions, but it does not fully address the challenge of graph-token understanding, calling for further improvement.

1. Introduction

Graph-structured data, an essential and prevalent form in the real world, plays a vital role in modeling interactions among objects, such as social networks (Guo & Wang, 2020), e-commerce networks (Zhang et al., 2022) and so

^{*}Equal contribution [†]Corresponding authors ¹Beijing University of Posts and Telecommunications ²China Telecom Bestpay ³Beihang University.

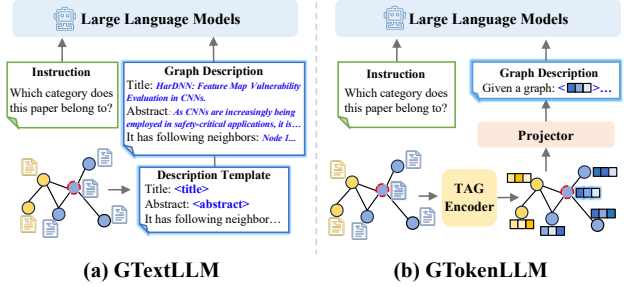


Figure 1. Two paradigms for applying LLMs to graph tasks.

on. Traditional message-passing graph neural networks (GNNs) (Kipf & Welling, 2017) are powerful tools for modeling graph structures, but they suffer from a weak multi-task handling capability (Chen et al., 2024a). Recently, the success of GPT-style LLMs (Touvron et al., 2023; Kojima et al., 2022) has motivated numerous studies to adapt them to graph learning, particularly for text-attributed graphs (TAGs), where each node is associated with a textual attribute. The ultimate goal is to build a graph foundation model that generalizes across diverse scenarios (Liu et al., 2025). In this setting, LLMs are typically employed as predictors to solve various graph tasks (e.g., node classification), showing promising potential and attracting considerable attention (Chen et al., 2024b; Fatemi et al., 2024; Kong et al., 2025; Zhang et al., 2025; Wang et al., 2025).

A central challenge in this endeavor is aligning graph data with the natural language embedding space so that LLMs can better comprehend graphs. To address this challenge, existing methods primarily follow two paradigms: Graph-Textualizing LLMs (GTextLLMs) (Tan et al., 2024b; Zhao et al., 2023; Chen et al., 2024b;c) and Graph-Tokenizing LLMs (GTokenLLMs) (Kong et al., 2025; Chen et al., 2024a; Tang et al., 2024). GTextLLMs utilize natural language to describe graphs via manually crafted prompt templates, directly aligning them with text, which is then combined with task instructions to query the LLM. As shown in Figure 1(a), for an academic TAG, node features and topology are formatted into a template description. This description includes a paper’s title/abstract and its neighbors’ titles. Although straightforward and easy to implement, GTextLLMs struggle to capture complex graph structures and long-range context in natural language, as shown by ex-

tensive benchmarks (Huang et al., 2023; Wang et al., 2024b), which raises doubts about their effectiveness.

In contrast, GTokenLLMs compress graph data with complex structures and lengthy text into the LLM input space, producing graph tokens that can be concatenated with natural-language instruction tokens to query the LLM. As shown in Figure 1(b), GTokenLLMs first encode the graph into embeddings using a TAG encoder (e.g., a GNN) to capture topological and textual information, and then transform these embeddings into the LLM space via a learned projector to yield graph tokens. As a result, a GTokenLLM represents the input as “Given a graph <graph>”, where “<graph>” is replaced with the resulting graph tokens. By compressing data into graph tokens, GTokenLLMs effectively mitigate the limitations of GTextLLMs and have been recognized as a mainstream paradigm in recent years. However, such highly compressed graph tokens may become the Achilles’ heel of GTokenLLMs, because they are projected from another space and do not correspond to any word in the LLM’s vocabulary, making their semantics opaque. Therefore, although GTokenLLMs have been widely recognized, the following question remains largely unknown:

Do GTokenLLMs fully understand graph tokens in the natural-language embedding space?

Understanding this question not only advances the development of reliable and explainable GTokenLLM applications, but also serves as a key touchstone for assessing the soundness and long-term viability of the GTokenLLM paradigm.

However, this question is challenging to answer. First, the lack of a unified, systematic framework for GTokenLLMs hinders the identification of model failures, preventing an in-depth understanding of the underlying causes. Second, there is a lack of evaluation tools or pipelines to comprehensively investigate this question. Current evaluations rely on simple, fixed instruction templates for each task, which inherently fail to explore the vast natural-language space.

In this paper, we first formalize a unified framework for GTokenLLMs by characterizing the stages of graph-data transformation, providing a clear and comprehensive view of this paradigm. Second, we propose **GTEval**, an evaluation pipeline that assesses graph-token understanding in GTokenLLMs through instruction transformations at the format and content levels. Finally, we conduct extensive experiments with GTEval to provide deeper insights into GTokenLLMs. The main observations are as follows:

- Existing GTokenLLMs do not fully understand graph tokens, exhibiting either over-sensitivity or over-insensitivity to instruction changes, and relying heavily on textual attributes for reasoning. Thus, they struggle to respond effectively to both format- and content-level instruction changes.
- Although graph tokens preserve task-relevant textual and

structural information in graph data and are attended to across all layers of the LLM, their utilization patterns differ across models and instruction variants.

- Extra instruction tuning improves performance on the original and seen instructions, but it does not fully address the challenge of graph-token understanding. Moreover, GTextLLMs underperform GTokenLLMs on the graph tasks despite stronger language understanding.

2. The Unified Framework of GTokenLLMs

2.1. The Formal Definition of GTokenLLM Framework

We systematically define the formal framework of GTokenLLMs, grounded in the observation that existing GTokenLLMs typically follow a common paradigm: graph data with complex structures and lengthy texts are encoded and aligned to the natural language embedding space, and then used as prefix tokens to query LLMs for downstream tasks. As shown in Figure 2, we characterize GTokenLLMs by the stages of graph data transformation, from raw graph inputs (Graph Input, GI) to encoded embeddings (Graph Embedding, GE), projected token representations (Graph Token, GT), and finally LLM-generated outputs (Text Output, TO).

Graph Input (GI). As input to GTokenLLMs, the text-attributed graph (TAG) can be viewed as a combination of graph structure and text attributes, where each node is associated with textual information. Formally, a TAG is defined as $\mathcal{G}=(\mathcal{V}, \mathcal{E}, \mathcal{T})$, where $\mathcal{V}=\{v_i\}_{i=1}^{|\mathcal{V}|}$, $\mathcal{E}=\{e_i\}_{i=1}^{|\mathcal{E}|}$, $\mathcal{T}=\{t_i\}_{i=1}^{|\mathcal{T}|}$ denote the node set, edge set, and text set, respectively. The adjacency matrix of $\mathcal{G}$ is denoted as $\mathbf{A} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$, where $\mathbf{A}_{ij}=1$ if nodes v_i and v_j are connected, and $\mathbf{A}_{ij}=0$ otherwise. Existing GTokenLLMs primarily consider the node classification task on TAGs. Specifically, each node v_i is associated with a textual label y_i indicating its category, and all such labels form the label set $\mathcal{Y}$. The goal is to predict the labels of nodes in the test set $\mathcal{V}_{\text{test}}$.

Stage 1: GI $\rightarrow$ GE. In the first stage, a TAG encoder is commonly used to compress complex graph structures and lengthy textual attributes into GEs $\mathbf{Z}=\{\mathbf{z}_i\}_{i=1}^{|\mathcal{V}|}$. Specifically, an off-the-shelf text encoder (e.g., BERT (Devlin et al., 2019)) is first used to encode the lengthy textual attributes, yielding the node features $\mathbf{X}=\{\mathbf{x}_i\}_{i=1}^{|\mathcal{V}|}$. Subsequently, structural features can be captured in two main ways. One approach is to use explicit structure embeddings derived from $\mathbf{A}$ as additional feature inputs (e.g., Laplacian embedding (Dwivedi & Bresson, 2020)), and then fuse them with $\mathbf{X}$ to form $\mathbf{Z}$ (Chen et al., 2024a; Ji et al., 2025). The other approach is to employ a GNN f_g , which implicitly captures structural information through message passing, i.e., $\mathbf{Z}=f_g(\mathbf{X}, \mathbf{A})$ (Wang et al., 2024a; Kong et al., 2025).

Stage 2: GE $\rightarrow$ GT. In this stage, GEs are mapped and aligned to the LLM token space via a learned projector,

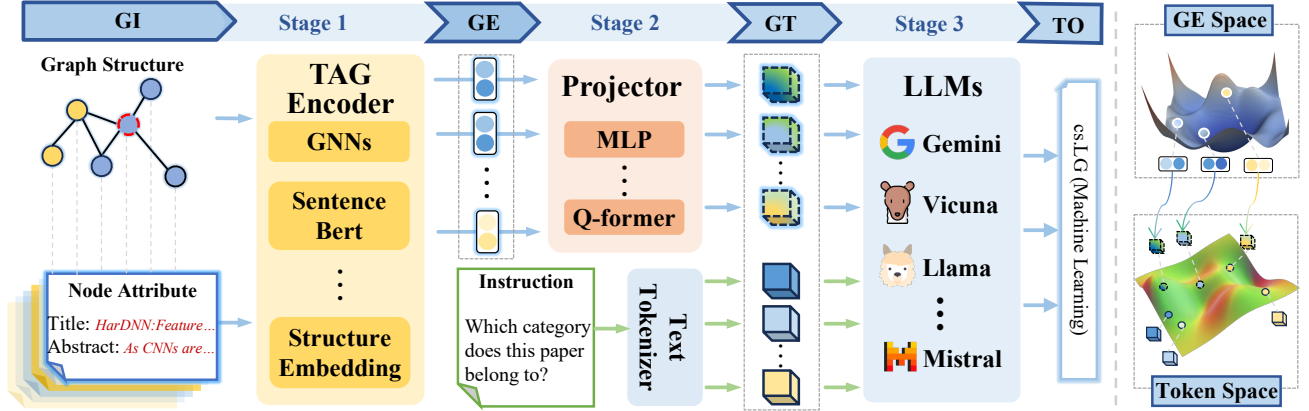


Figure 2. The GTokenLLM framework, illustrating a stage-wise transformation of graph data from raw graph inputs (GI) to graph embeddings (GE), projected graph tokens (GT), and finally LLM-generated textual outputs (TO).

yielding GTs. Since GEs and language tokens reside in distinct spaces in terms of both feature dimensions and semantics, GEs are not directly compatible with LLMs for querying, unlike textual tokens. To bridge this gap, GTokenLLMs train a projector f_p to map $\mathbf{Z}$ into GTs, i.e., $\mathbf{T} = f_p(\mathbf{Z})$, using graph tasks (e.g., node classification).

Stage 3: GT $\rightarrow$ TO. In the final stage, GTs are concatenated with other textual content, mainly task instructions, to query the LLM and generate TOs $\hat{y}$. This process relies on a manually designed prompt template that describes the graph, where special placeholders are predefined and later replaced by GTs during tokenization. The exact form of the template depends on the prompting strategy, such as where GTs are placed and how structural information is expressed. For example, a prompt template may take the form “Given a citation graph $\langle \text{graph} \rangle$. Which category does this paper belong to?”, where $\langle \text{graph} \rangle$ is a placeholder replaced by the GTs of the central node and its neighbors.

In this paper, we investigate 6 representative GTokenLLMs: LLaGA (Chen et al., 2024a), InstructGLM (Ye et al., 2023), GraphGPT (Tang et al., 2024), GraphTranslator (Zhang et al., 2024), TEA-GLM (Wang et al., 2024a), and GOFA (Kong et al., 2025). We show that these models can be fitted into our proposed framework. Given the limited space, please refer to Appendix B for more details.

3. GTEval: Evaluating the Graph Token Understanding of GTokenLLMs

3.1. The Definition of GTEval

We characterize the understanding of graph tokens at two levels. At the format level, understanding requires that GTokenLLMs are not only fitted to specific task instructions in the training set, but can also appropriately respond to format changes, such as task paraphrasing. At the content level, understanding means that GTokenLLMs can generalize to new task instructions with different semantic content, given the

same GT inputs. Therefore, we design four evaluation methods in GTEval by transforming the original task instructions at both the format and content levels, to assess format-level (Rephrasing) and content-level (Reversing, Relabeling, and Randomizing) understanding of GTs in GTokenLLMs.

Original task instruction. Current evaluations of GTokenLLMs rely on a single, fixed instruction template for each task. As illustrated in Table 1, the first box presents the textual features of an example node belonging to the class “cs.LG (Machine Learning).” The second box shows the original instruction template used during both training and evaluation. In this template, the black text is fixed, while the blue text is instantiated with GEs and label lists. In practice, since GTokenLLMs output free-form text, **Accuracy (Acc $\uparrow$)** is computed by counting a prediction as correct if it contains the ground-truth label and no other label (Chen et al., 2024a). Formally, accuracy is defined as:

$$\text{Accuracy} = \frac{1}{|\mathcal{V}_{\text{test}}|} \sum_{y_i \in \mathcal{V}_{\text{test}}} \mathbb{I}(\text{contains}(\hat{y}_i, y_i) \wedge \forall y \in \mathcal{Y} \setminus \{y_i\}, \neg \text{contains}(\hat{y}_i, y)), \quad (1)$$

where $\hat{y}_i$ denotes the textual output generated by the LLM, $y_i \in \mathcal{Y}$ is the ground-truth textual label, $\text{contains}(\hat{y}_i, y)$ indicates that the output text $\hat{y}_i$ contains the label string y , and $\mathbb{I}(\cdot)$ is the indicator function.

Format-level modification. A basic requirement for GTokenLLMs is to produce consistent responses for the same GTs when the task format changes but the underlying semantics remain the same. To evaluate this property, we define **Rephrasing** as paraphrasing the instruction while keeping the task itself (e.g., node classification) unchanged. Minor edits, such as synonym replacement or word reordering, are allowed in task-irrelevant parts. As shown in the third box of Table 1, terms such as “Given,” “node-centered,” and “classify ... into 40 classes” are rephrased as “Using,” “node-centric,” and “classify ... into one of the 40 given categories,” respectively. A GTokenLLM that fails to properly

Table 1. Illustrative examples of instruction types instantiated for the LLaGA in GTEval, with key modifications underlined>.

Node ID: 43800	Title: HarDNN: Feature Map Vulnerability Evaluation in CNNs.
Abstract: As CNNs are increasingly being employed in safety-critical applications, it is important that they behave reliably ... This paper presents HarDNN, a software-directed approach ...	Label: cs.LG(Machine Learning)
Instruction Type: Original	
Instruction Template: Given a node-centered graph: $\langle \text{graph} \rangle$, we need to classify the center node into 40 classes: $\langle \text{cs.LG (Machine Learning)}, \text{cs.AI(Artificial Intelligence)}, \dots, \text{cs.CR (Cryptography and Security)} \rangle$, please tell me which class the center node belongs to?	
Expected Output: cs.LG (Machine Learning).	Possible Output (✓): cs.LG (Machine Learning).
Instruction Type: Rephrasing	
Instruction Template: Using the <u>node-centric</u> graph: $\langle \text{graph} \rangle$, classify the central node into <u>one of the 40 given categories</u> : $\langle \text{cs.LG (Machine Learning)}, \text{cs.AI(Artificial Intelligence)}, \dots, \text{cs.CR(Cryptography and Security)} \rangle$.	
Expected Output: cs.LG (Machine Learning).	Possible Output (✗): cs.CR (Cryptography and Security).
Instruction Type: Relabeling	
Instruction Template: Using the node-centric graph: $\langle \text{graph} \rangle$, classify the central node into one of the <u>given categories</u> : $\langle \text{Artificial Intelligence and Machine Learning, Theoretical Computer Science, ..., Applied Computing and Engineering} \rangle$.	
Expected Output: Artificial Intelligence and Machine Learning.	Possible Output (✗): Applied Computing and Engineering.
Instruction Type: Reversing	
Instruction Template: Using the node-centric graph: $\langle \text{graph} \rangle$, classify the central node into the <u>least probable</u> of the 40 given categories: $\langle \text{cs.LG (Machine Learning)}, \text{cs.AI (Artificial Intelligence)}, \dots, \text{cs.CR (Cryptography and Security)} \rangle$.	
Expected Output: cs.CR (Cryptography and Security).	Possible Output (✗): cs.LG (Machine Learning).
Instruction Type: Randomizing	
Instruction Template: Using the node-centric graph: $\langle \text{graph} \rangle$, the ducks are planning to organize a concert in the park: $\langle \text{cs.LG (Machine Learning)}, \text{cs.AI(Artificial Intelligence)}, \dots, \text{cs.CR (Cryptography and Security)} \rangle$.	
Expected Output: Any answer that is not related to node classification.	Possible Output (✗): cs.LG (Machine Learning).

understand GTs at the format level may produce inconsistent predictions under such rephrased instructions. Notably, since the task semantics remain unchanged, we use the accuracy defined in Equation 1 as the evaluation metric.

Content-level modification. A higher expectation is that GTokenLLMs respond correctly to changes in task content given the same GT inputs. Here, we define three modification methods. “Relabeling” and “Reversing” evaluate the model’s adaptability to changes in label space and semantics; in these settings, the model should adapt to the new instructions with GTs, as the tasks are similar but differ in labels or structure. “Randomizing” assesses the worst-case scenario by introducing random task instructions. If models are overfitted to training tasks, they may still produce answers for the original task despite the random instructions.

• **Relabeling:** This instruction replaces the original label space with a human-defined one that is human-readable and reasonable. As illustrated in the fourth box of Table 1, the original label space “cs.LG (Machine Learning), cs.AI (Artificial Intelligence), ..., cs.CR (Cryptography and Security)” is replaced with “Artificial Intelligence and Machine Learning, Information Systems and Data Management, ..., Applied Computing and Engineering”. This transition is achieved by summarizing the original labels. For instance, “cs.LG (Machine Learning)” becomes “Artificial Intelligence and Machine Learning”. If GTokenLLMs fail to capture the intrinsic semantics of the original labels, they may be unable to produce the correct relabeled outputs, leading to incorrect or even invalid predictions. Formally, we further define the **Relabel** ($\uparrow$) metric to evaluate models:

$$\text{Relabel} = \frac{1}{|\mathcal{V}_{\text{test}}|} \sum_{v_i \in \mathcal{V}_{\text{test}}} \mathbb{I} \left(\text{contains}(\hat{y}_i, y_i^{\text{rel}}) \wedge \forall y \in \mathcal{Y}^{\text{rel}} \setminus \{y_i^{\text{rel}}\}, \neg \text{contains}(\hat{y}_i, y) \wedge \forall y \in \mathcal{Y}, \neg \text{contains}(\hat{y}_i, y) \right), \quad (2)$$

where $y_i^{\text{rel}} \in \mathcal{Y}^{\text{rel}}$ is the relabeled ground-truth label of node v_i , and $\mathcal{Y}^{\text{rel}}$ denotes the relabeled label set.

• **Reversing:** Unlike Relabeling, this setting reverses the objective of the original task, forcing the model to break its default preference. As illustrated in the fifth box of Table 1, the phrase “least probable” is added to require the model to identify the most unrelated category instead of the most relevant one. If GTokenLLMs fail to understand the semantics of graph tokens, they may be stuck at memorizing label names rather than performing discriminative reasoning, and thus still output the original label. We further define the **Reverse** ($\uparrow$) metric to evaluate this capability as follows:

$$\mathcal{V}_{\text{corr}} = \left\{ v_i \in \mathcal{V}_{\text{test}} \mid \text{contains}(\hat{y}_i^{\text{ori}}, y_i) \wedge \forall y \in \mathcal{Y} \setminus \{y_i\}, \neg \text{contains}(\hat{y}_i^{\text{ori}}, y) \right\}, \quad (3)$$

$$\text{Reverse} = 1 - \frac{1}{|\mathcal{V}_{\text{corr}}|} \sum_{v_i \in \mathcal{V}_{\text{corr}}} \mathbb{I} \left(\text{contains}(\hat{y}_i^{\text{rev}}, y_i) \wedge \forall y \in \mathcal{Y} \setminus \{y_i\}, \neg \text{contains}(\hat{y}_i^{\text{rev}}, y) \right), \quad (4)$$

where $\hat{y}_i^{\text{ori}}$ and $\hat{y}_i^{\text{rev}}$ are the textual outputs generated by the GTokenLLM under the original and reversing instructions for node v_i , respectively.

Table 2. Evaluation results of GTokenLLMs on GTEval. Δ denotes the change from the original, and $\emptyset$ indicates illegal outputs.

Model	Dataset	Format-Level			Content-Level		
		Ori. Acc ($\uparrow$)	Rep. Acc ($\uparrow$)	Δ	Relabel ($\uparrow$)	Reverse ($\uparrow$)	Random ($\uparrow$)
LLaGA	Cora	87.45	86.44 $\pm$ 0.93	$\downarrow$ 1.16%	5.74 $\pm$ 3.90	1.73 $\pm$ 0.95	0.85 $\pm$ 0.54
	Pubmed	92.13	92.39 $\pm$ 0.38	$\uparrow$ 0.28%	40.30 $\pm$ 7.89	1.48 $\pm$ 0.55	1.93 $\pm$ 2.68
	Arxiv	74.36	69.55 $\pm$ 2.34	$\downarrow$ 6.47%	1.19 $\pm$ 0.49	4.76 $\pm$ 0.75	5.59 $\pm$ 1.65
	Products	83.22	81.79 $\pm$ 1.09	$\downarrow$ 1.72%	6.84 $\pm$ 0.85	2.84 $\pm$ 1.39	3.76 $\pm$ 0.30
	Overall	84.29	81.54	$\downarrow$ 3.26%	13.52	2.70	3.03
InstructGLM	Cora	78.59	39.24 $\pm$ 19.71	$\downarrow$ 50.07%	$\emptyset$	15.81 $\pm$ 15.25	$\emptyset$
	Pubmed	91.87	55.05 $\pm$ 22.67	$\downarrow$ 40.08%	$\emptyset$	9.46 $\pm$ 10.66	$\emptyset$
	Arxiv	76.85	74.96 $\pm$ 1.32	$\downarrow$ 2.45%	$\emptyset$	18.63 $\pm$ 8.33	$\emptyset$
	Overall	82.44	56.42	$\downarrow$ 31.56%	$\emptyset$	14.63	$\emptyset$
GraphGPT	Cora-70	18.66	18.40 $\pm$ 1.25	$\downarrow$ 1.41%	30.26 $\pm$ 1.04	74.51 $\pm$ 10.62	89.83 $\pm$ 1.93
	Pubmed	80.71	74.21 $\pm$ 6.39	$\downarrow$ 8.05%	57.74 $\pm$ 1.90	40.63 $\pm$ 18.07	58.27 $\pm$ 4.19
	Arxiv	62.50	39.80 $\pm$ 12.26	$\downarrow$ 36.32%	21.46 $\pm$ 1.37	27.68 $\pm$ 17.84	99.00 $\pm$ 0.27
	Overall	53.96	44.14	$\downarrow$ 18.20%	36.49	47.61	82.37
TEA-GLM	Cora-70	12.10	10.06 $\pm$ 1.28	$\downarrow$ 16.84%	21.46 $\pm$ 5.99	34.84 $\pm$ 14.89	76.71 $\pm$ 3.66
	Pubmed	79.54	59.09 $\pm$ 19.40	$\downarrow$ 25.71%	60.33 $\pm$ 24.66	60.97 $\pm$ 25.76	40.05 $\pm$ 4.79
	History	50.76	38.05 $\pm$ 7.36	$\downarrow$ 25.03%	24.05 $\pm$ 5.93	70.75 $\pm$ 16.66	64.16 $\pm$ 5.03
	Overall	47.47	35.73	$\downarrow$ 24.73%	35.28	55.52	60.31
GOFA	Cora	68.91	67.70 $\pm$ 0.79	$\downarrow$ 1.75%	72.49 $\pm$ 25.48	2.03 $\pm$ 0.87	4.48 $\pm$ 4.86
	WikiCS	71.06	69.77 $\pm$ 0.54	$\downarrow$ 1.81%	76.63 $\pm$ 0.84	3.97 $\pm$ 2.67	3.77 $\pm$ 1.30
	Overall	69.98	68.74	$\downarrow$ 1.78%	74.56	3.00	4.13
GraphTranslator	Arxiv	26.59	16.02 $\pm$ 2.87	$\downarrow$ 39.73%	20.90 $\pm$ 3.65	70.71 $\pm$ 9.37	91.11 $\pm$ 8.63

• **Randomizing:** This modification considers an extreme case where the original task instruction is replaced with an irrelevant sentence, to test whether a GTokenLLM can distinguish valid task instructions from unrelated text. Under this setting, the model is expected to refrain from producing task-specific outputs. However, a GTokenLLM that fails to distinguish the task itself may still respond as if performing the original task. As illustrated in the sixth box of Table 1, when the node classification instruction is replaced by an irrelevant sentence such as “the ducks are planning to organize a concert in the park,” the model may still output the original classification result, e.g., “cs.LG (Machine Learning).” Here, we further define the **Random** ($\uparrow$) metric:

$$\text{Random} = \frac{1}{|\mathcal{V}_{\text{test}}|} \sum_{v_i \in \mathcal{V}_{\text{test}}} \mathbb{I}(\forall y \in \mathcal{Y}, \neg \text{contains}(\hat{y}_i, y)). \quad (5)$$

3.2. Evaluation Settings of GTEval

In this paper, we evaluate 6 representative GTokenLLMs, including LLaGA (Chen et al., 2024a), InstructGLM (Ye et al., 2023), GraphGPT (Tang et al., 2024), GraphTranslator (Zhang et al., 2024), TEA-GLM (Wang et al., 2024a), and GOFA (Kong et al., 2025). To more faithfully reflect the models’ performance in practice, we directly use the released checkpoint weights and evaluate each model on its corresponding test set, without additional training or fine-tuning. The dataset statistics used by different models

are summarized in Table 8. To construct test instructions efficiently and reliably, we design four instruction modification templates with the assistance of ChatGPT, followed by manual verification by domain experts. For each dataset and model, we generate ten variants per template to mitigate randomness introduced by wording variations. These instruction variants are then instantiated with the test data for evaluation. We report the mean and standard deviation of the results across different instruction modification templates. More experiment details are provided in Appendix D.

4. Experimental Results on GTEval

In this section, we conduct extensive experiments with GTEval to explore whether GTokenLLMs fully understand GTs. The results indicate that their capabilities in this regard remain at a preliminary stage. To better understand the underlying reasons, we further perform a series of analytical experiments that inspect each stage of the GTokenLLM framework defined in Section 2. Finally, we explore two practical directions to alleviate this limitation: instruction tuning on template variants and directly adopting GTextLLMs.

4.1. Overall Performance on GTEval

Table 2 reports the results of 6 representative GTokenLLMs evaluated with GTEval. We have the following observations:

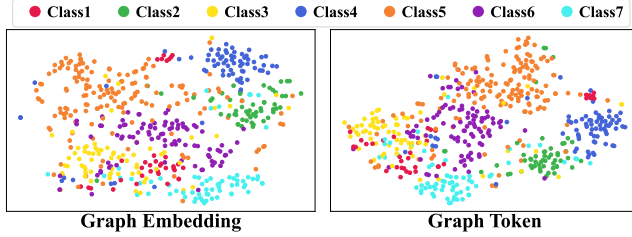


Figure 3. t-SNE visualization of GEs and GTs on Cora.

Observation 1: GTokenLLMs fail to exhibit consistent understanding of GTs across different instruction formulations, struggling to effectively respond to both format- and content-level changes. No existing GTokenLLM performs well across all instruction variants. Specifically, these models achieve around 70%–90% accuracy under the original instructions. For format changes, Rephrasing results in much lower accuracy, with gaps reaching up to 50%. For content changes, the performance of most methods is below 50% and even degrades to under 10%. InstructGLM even produces invalid outputs such as blank responses or “yes”. Overall, these results indicate that current GTokenLLMs still exhibit a clearly limited understanding of graph tokens.

Observation 2: GTokenLLMs exhibit over-insensitivity or over-sensitivity behavior under instruction changes. For example, both LLaGA and GOFA remain stable under the format-level change (Rephrasing), but achieve performance below 6% under content-level changes (Reversing and Randomizing). This suggests that the models may be severely over-fitted to the training tasks, showing “over-insensitivity” to natural language instructions. In contrast, GraphGPT, TEA-GLM, and GraphTranslator exhibit “over-sensitivity” to format-level changes, with overall accuracy variations of approximately 18%–40% under Rephrasing, while showing comparatively better results on content-level changes. InstructGLM exhibits over-sensitivity to both format-level and content-level changes, with low accuracy on format changes and invalid outputs on content changes.

Summary: Existing GTokenLLMs exhibit only a preliminary understanding of GTs, which remains insufficient for full comprehension. We call for future evaluations not to be restricted to a single fixed task instruction, but to consider diverse task formulations for more reliable assessments.

4.2. Deeper Insights into Each Stage of GTokenLLMs

To dive into understanding the underlying reasons, we conduct a stage-wise analysis of GTokenLLMs, following the transformation from GIs to GEs, GTs, and finally TOs.

4.2.1. INSIGHTS FROM “GI-TO-GE” STAGE

First, to examine how GIs (including textual attributes and graph structure) affect GTokenLLM inference, we apply

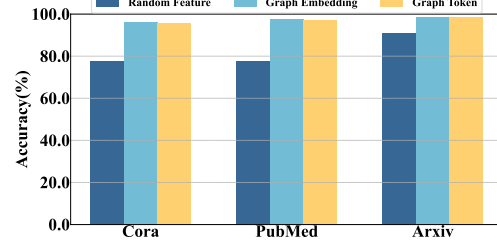


Figure 4. Link prediction with GEs, GTs, and random features.

 Table 3. Model performance under the original and PRBCD structure attack, where Δ denotes the change from the original.

Model	Dataset	Original	PRBCD	Δ
GCN	Cora	89.85	23.43	$\downarrow 73.92\%$
	Cora-70	19.15	3.03	$\downarrow 84.18\%$
	Pubmed	80.88	23.58	$\downarrow 70.85\%$
	Overall	63.30	16.70	$\downarrow 73.65\%$
LLaGA	Cora	87.45	48.71	$\downarrow 44.30\%$
	Pubmed	92.13	60.29	$\downarrow 34.56\%$
	Overall	89.79	54.40	$\downarrow 39.30\%$
GraphGPT	Cora-70	18.66	20.99	$\uparrow 12.49\%$
	Pubmed	80.71	80.46	$\downarrow 0.31\%$
	Overall	49.69	50.73	$\uparrow 2.09\%$
TEA-GLM	Cora-70	12.10	12.39	$\uparrow 2.40\%$
	Pubmed	79.54	79.21	$\downarrow 0.41\%$
	Overall	45.82	45.80	$\downarrow 0.04\%$

PRBCD (Geisler et al., 2021) to attack graph topology. The perturbation budget for each node is its degree, enabling effective structural disruption. Table 3 reports the accuracy of vanilla GCN (Kipf & Welling, 2017), LLaGA, GraphGPT, and TEA-GLM. Notably, LLaGA relies solely on GTs during inference, whereas GraphGPT and TEA-GLM additionally incorporate the textual attributes of the central node.

Observation 3: GTokenLLMs lack an effective understanding of GTs and heavily rely on textual attributes for reasoning. As a structure-dependent baseline, GCN suffers a severe performance drop on all datasets (e.g., 73.65% overall). In contrast, although LLaGA also degrades substantially (39.30% overall), it shows “over-insensitivity” to content-level modifications (Table 2), suggesting that it may mainly memorize a direct mapping from GTs to labels rather than understand them. Meanwhile, GraphGPT and TEA-GLM remain almost unchanged and even slightly improve in some cases, suggesting that their reasoning is mainly based on the input node text rather than GT understanding.

4.2.2. INSIGHTS FROM “GE-TO-GT” STAGE

To validate whether GTs retain sufficient textual information, we visualize the embedding spaces of GEs and GTs in LLaGA using t-SNE (Figure 3). To assess structural information, we train separate GCNs for link prediction using GEs, GTs, and randomly initialized features (Figure 4).

Table 4. The performance of instruction-tuned GTokenLLMs with instruction variants Rephrasing and Randomizing. “Imp.” denotes the improvement over the corresponding result in Table 2 (i.e., without extra instruction tuning).

Model	Dataset	Format-Level				Content-Level					
		Ori. Acc (↑)	Imp.	Rep. Acc (↑)	Imp.	Relabel (↑)	Imp.	Reverse (↑)	Imp.	Random (↑)	Imp.
LLaGA	Cora	87.64	↑0.22%	87.36±0.46	↑1.06%	90.54±0.60	↑1477.35%	1.14±0.53	↓34.10%	0.00±0.00	↓100.00%
	Pubmed	95.18	↑3.31%	94.85±0.24	↑2.66%	96.29±0.30	↑138.93%	0.56±0.29	↓62.16%	0.00±0.00	↓100.00%
	Arxiv	75.61	↑1.68%	75.46±0.12	↑8.50%	82.58±0.23	↑6839.50%	2.99±0.21	↓37.18%	0.00±0.00	↓100.00%
	Overall	86.14	↑1.76%	85.89	↑3.74%	89.89	↑470.42%	1.56	↓41.15%	0.00	↓100.00%
GraphGPT	Cora-70	20.41	↑9.38%	18.92±1.23	↑2.83%	35.92±1.84	↑18.70%	59.20±7.33	↓20.55%	83.24±1.73	↓7.34%
	Pubmed	89.85	↑11.32%	88.76±1.37	↑19.61%	91.52±0.72	↑58.50%	19.60±16.37	↓51.76%	16.40±7.03	↓71.86%
	Arxiv	66.40	↑6.24%	67.19±0.76	↑68.82%	46.24±1.02	↑115.47%	4.41±0.65	↓84.07%	99.52±0.16	↑0.53%
	Overall	58.89	↑9.14%	58.29	↑32.07%	57.89	↑58.67%	27.74	↓41.74%	66.39	↓19.40%

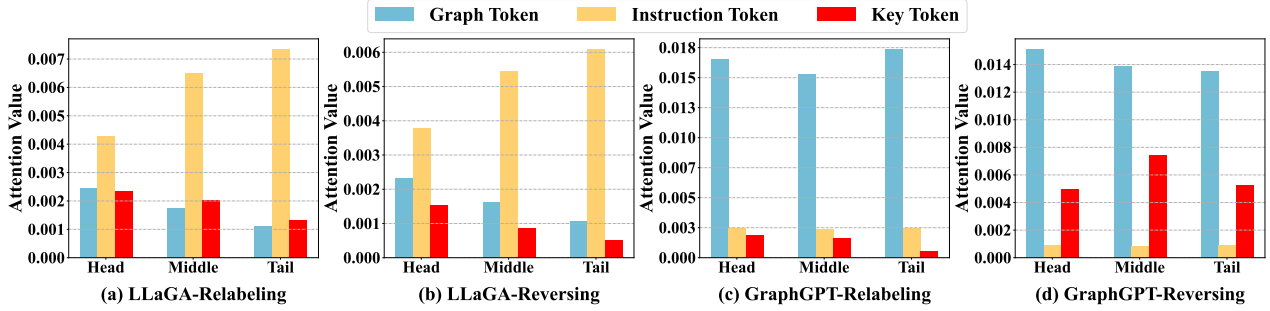


Figure 5. Attention distribution of GTokenLLMs under Relabeling and Reversing instruction variants on Cora.

Observation 4: GTs preserve task-relevant textual and structural information in graph data. Figure 3 suggests that GTs retain textual signals: nodes with the same label cluster more tightly, and the class boundaries are clearer than those formed by GEs, indicating that supervision from downstream tasks further shapes GTs toward community-level textual cues. Figure 4 suggests that GTs also retain structural signals. In link prediction, a GCN trained on GTs performs comparably to one trained on GEs, and both outperform randomly initialized features, implying that edge-level information is largely preserved. The small gap may reflect that GTs encode classification-oriented signals that are not fully aligned with link prediction. Overall, GTs retain both textual and structural information, especially that related to downstream tasks.

4.2.3. INSIGHTS FROM “GT-TO-TO” STAGE

To examine whether the LLM attends to graph tokens during inference, we analyze attention weights across layers. Following (Darcet et al., 2023; Dosovitskiy, 2020), we extract the cross-attention matrices between output and input tokens from the first, middle, and last three layers (denoted as *head*, *middle*, and *tail*) and average them over attention heads and layers. We then group the attention weights by token type: graph tokens, instruction tokens, and key tokens. Key tokens are the words that distinguish an instruction variant

from the original one (e.g., “least probable” in Reversing). Figure 5 shows the results for LLaGA and GraphGPT under Relabeling and Reversing on the Cora dataset.

Observation 5: GTs are attended to across all layers of the LLM, with attention patterns differing across models and instruction variants. Across all layers, LLaGA and GraphGPT assign non-trivial attention to GTs, suggesting that GTs are not ignored when generating TOs. However, the attention distribution differs by model and instruction type. Specifically, LLaGA allocates most attention to instruction tokens and the least to key tokens. This pattern may bias the model toward the seen training templates and weaken its adaptation to content-level variants. In contrast, GraphGPT allocates comparable attention to key tokens under Relabeling and even higher attention under Reversing, aligning with its better adaptation to content-level variants.

4.3. Is Instruction Tuning a Straightforward Remedy?

Instruction tuning has been acknowledged as an intuitive solution for generalizing to unseen instructions (Zhang et al., 2026; Kung et al., 2023). To this end, we construct additional instruction tuning sets. Since ground-truth are unavailable for Randomizing and Reversing, we only add variants of Rephrasing and Relabeling. For each model, we generate 10 templates per variant and instantiate them using the original training data. This setup keeps the training set

Table 5. Evaluation results of the GTextLLM (Chen et al., 2024b) on GTEval. Δ denotes the change from the original.

Model	Dataset	Format-Level			Content-Level		
		Ori. Acc ($\uparrow$)	Rep. Acc ($\uparrow$)	Δ	Relabel ($\uparrow$)	Reverse ($\uparrow$)	Random ($\uparrow$)
Zero-Shot	Cora	67.15	68.41 $\pm$ 0.91	$\uparrow$ 1.88%	58.76 $\pm$ 2.69	92.92 $\pm$ 10.89	97.40 $\pm$ 1.12
	Pubmed	78.93	82.28 $\pm$ 3.11	$\uparrow$ 4.24%	74.90 $\pm$ 16.25	84.03 $\pm$ 17.36	99.44 $\pm$ 0.36
	Arxiv	54.20	52.62 $\pm$ 2.27	$\downarrow$ 2.92%	47.12 $\pm$ 1.36	91.86 $\pm$ 15.89	95.50 $\pm$ 1.50
	Overall	66.76	67.77	$\uparrow$ 1.51%	60.26	89.60	97.45
1-Hop Title+Label	Cora	69.37	80.83 $\pm$ 1.71	$\uparrow$ 16.52%	80.66 $\pm$ 1.52	85.15 $\pm$ 9.13	91.16 $\pm$ 5.41
	Pubmed	88.07	86.60 $\pm$ 4.43	$\downarrow$ 1.67%	70.65 $\pm$ 4.92	89.14 $\pm$ 3.87	100.00 $\pm$ 0.00
	Arxiv	47.40	60.25 $\pm$ 1.95	$\uparrow$ 27.11%	59.82 $\pm$ 3.05	93.87 $\pm$ 8.71	97.44 $\pm$ 3.92
	Overall	68.28	75.89	$\uparrow$ 11.15%	70.38	89.39	96.20

 Table 6. GTokenLLM with GPT-4o compared to GPT-3.5 on Cora. Δ denotes the change from the original.

Model	Format-Level			Content-Level		
	Ori. Acc ($\uparrow$)	Rep. Acc ($\uparrow$)	Δ	Relabel ($\uparrow$)	Reverse ($\uparrow$)	Random ($\uparrow$)
GPT-3.5	67.15	68.41 $\pm$ 0.91	$\uparrow$ 1.88%	58.76 $\pm$ 2.69	92.92 $\pm$ 10.89	97.40 $\pm$ 1.12
GPT-4o	70.11	69.00 $\pm$ 1.11	$\downarrow$ 1.58%	64.66 $\pm$ 1.84	93.16 $\pm$ 5.42	94.39 $\pm$ 1.20

size unchanged while increasing instruction diversity during training. We apply it to two representative GTokenLLMs, LLaGA and GraphGPT, and report the results in Table 4.

Observation 6: Extra instruction tuning improves performance under the original task instruction. Although we do not introduce additional training data, enriching the instruction templates during tuning leads to higher accuracy under the original task instruction. This gain may come from improved instruction-following capability on GT inputs, resulting in more accurate textual predictions. These results suggest that increasing instruction diversity is a simple and practical enhancement during instruction tuning.

Observation 7: Variant-based instruction tuning improves performance on seen variants but does not generalize to unseen ones. The tuned models consistently outperform the original models on the two seen variant types, indicating that instruction tuning is effective when the instruction patterns are covered during training. However, performance shows little improvement and can even degrade on the two unseen types (Randomizing and Reversing), indicating that instruction tuning cannot fundamentally address the deficiency of GTokenLLMs in understanding GTs. This observation also encourages researchers to strive for GTokenLLMs with stronger graph token understanding.

4.4. Is GTextLLM an Alternative to GTokenLLMs?

Unlike GTokenLLMs, which learn to align graph tokens with textual tokens, GTextLLM describes graphs in natural language via manually crafted prompt templates, directly aligning graph inputs with text. For a comprehensive comparison, we evaluate GTextLLM under GTEval and report the results in Table 5, using the prompt templates adopted from Chen et al. (Chen et al., 2024b). For consistency with

prior GTokenLLM studies, we use GPT-3.5 as the default GTextLLM backbone. We also report GPT-4o results on Cora with the zero-shot prompt in Table 6.

Observation 8: GTextLLMs underperform GTokenLLMs on the same graph task, even though their language understanding is better. This gap is expected because GTextLLMs solve graph tasks in a zero-shot, prompt-based manner, whereas GTokenLLMs learn specialized graph representations (e.g., GTs) tailored to downstream supervision. As a result, GTokenLLMs retain a clear advantage in task accuracy (Chen et al., 2024b;a). Moreover, GPT-4o achieves higher accuracy under the original prompt than GPT-3.5, but still falls short of GTokenLLMs (87.45%). Meanwhile, GPT-4o and GPT-3.5 show similar behavior on GTEval. Overall, these results support the same conclusion across backbones: using natural-language graph descriptions avoids the graph-token understanding issue by construction, yet GTextLLMs remain substantially less competitive than specialized GTokenLLMs on graph tasks.

5. Conclusion

Recently, GTokenLLMs have emerged as a mainstream paradigm for extending LLMs to TAGs. In this work, we first systematically examine whether these models fully understand graph tokens. By formalizing GTokenLLMs and introducing GTEval, an evaluation pipeline based on format- and content-level modifications, we assess their ability to adapt to task variations. Our results show that existing GTokenLLMs exhibit only a preliminary stage of graph-token understanding and remain far from reliable semantic comprehension. We hope this study encourages more careful evaluation and deeper reflection on graph-token understanding in future research.

Impact Statement

This paper advances the field of machine learning by examining the limitations of graph-tokenizing large language models (GTokenLLMs). Our work aims to improve their reliability and explainability, with potential applications in social network analysis, healthcare, and e-commerce. Moreover, our work serves as a key touchstone for assessing the soundness and long-term viability of the GTokenLLM paradigm. While we do not anticipate immediate ethical risks, it remains important to monitor the deployment of such models to ensure fairness and transparency, particularly in high-stakes applications. Continued refinement of GTokenLLMs is necessary to mitigate potential biases and promote responsible use across diverse settings.

References

- Chen, R., Zhao, T., JAISWAL, A. K., et al. Llava: Large language and graph assistant. In *Forty-first International Conference on Machine Learning*, 2024a.
- Chen, Z., Mao, H., Li, H., et al. Exploring the potential of large language models (llms) in learning on graphs. *ACM SIGKDD Explorations Newsletter*, 25(2):42–61, 2024b.
- Chen, Z., Mao, H., Wen, H., et al. Label-free node classification on graphs with large language models (llms). In *The Twelfth International Conference on Learning Representations*, 2024c.
- Chiang, W.-L., Li, Z., Lin, Z., et al. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023. URL <https://lmsys.org/blog/2023-03-30-vicuna/>.
- Dai, X., Qu, H., Shen, Y., et al. How do large language models understand graph patterns? a benchmark for graph pattern comprehension. *arXiv preprint arXiv:2410.05298*, 2024.
- Darcet, T., Oquab, M., Mairal, J., et al. Vision transformers need registers. *arXiv preprint arXiv:2309.16588*, 2023.
- Devlin, J., Chang, M.-W., Lee, K., et al. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186, 2019.
- Dosovitskiy, A. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- Duan, K., Liu, Q., Chua, T.-S., et al. Simteg: A frustratingly simple approach improves textual graph learning. *arXiv preprint arXiv:2308.02565*, 2023.
- Dwivedi, V. P. and Bresson, X. A generalization of transformer networks to graphs. *arXiv preprint arXiv:2012.09699*, 2020.
- Fatemi, B., Halcrow, J., and Perozzi, B. Talk like a graph: Encoding graphs for large language models. In *The Twelfth International Conference on Learning Representations*, 2024.
- Ge, T., Jing, H., Wang, L., et al. In-context autoencoder for context compression in a large language model. In *The Twelfth International Conference on Learning Representations*, 2024.
- Geisler, S., Schmidt, T., Şirin, H., et al. Robustness of graph neural networks at scale. *Advances in Neural Information Processing Systems*, 34:7637–7649, 2021.
- Guo, J., Du, L., Liu, H., et al. Gpt4graph: Can large language models understand graph structured data? an empirical evaluation and benchmarking. *arXiv preprint arXiv:2305.15066*, 2023.
- Guo, K., Liu, Z., Chen, Z., et al. Learning on graphs with large language models (llms): A deep dive into model robustness. *arXiv preprint arXiv:2407.12068*, 2024.
- Guo, Z. and Wang, H. A deep graph neural network-based mechanism for social recommendations. *IEEE Transactions on Industrial Informatics*, 17(4):2776–2783, 2020.
- Hamilton, W., Ying, Z., and Leskovec, J. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.
- Harris, Z. S. Distributional structure. *Word*, 10(2-3):146–162, 1954.
- Hu, E. J., Shen, Y., Wallis, P., et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- Hu, W., Fey, M., Zitnik, M., et al. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33:22118–22133, 2020.
- Huang, J., Zhang, X., Mei, Q., et al. Can llms effectively leverage graph structural information: when and why. *arXiv preprint arXiv:2309.16595*, 2023.
- Ji, Y., Liu, C., Chen, X., et al. From anchors to answers: A novel node tokenizer for integrating graph structure into large language models. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pp. 1124–1134, 2025.
- Kipf, T. N. and Welling, M. Semi-supervised classification with graph convolutional networks. In *5th International Conference on Learning Representations, ICLR 2017*,

- Toulon, France, April 24-26, 2017, *Conference Track Proceedings*. OpenReview.net, 2017. URL <https://openreview.net/forum?id=SJU4ayYgl>.
- Kojima, T., Gu, S. S., Reid, M., et al. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213, 2022.
- Kong, L., Feng, J., Liu, H., et al. Gofa: A generative one-for-all model for joint graph language modeling. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Kung, P.-N., Yin, F., Wu, D., et al. Active instruction tuning: Improving cross-task generalization by training on prompt sensitive tasks. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 1813–1829, 2023.
- Li, X., Chen, W., Chu, Q., et al. Can large language models analyze graphs like professionals? a benchmark, datasets and models. *arXiv preprint arXiv:2409.19667*, 2024.
- Liu, J., Yang, C., Lu, Z., Chen, J., Li, Y., Zhang, M., Bai, T., Fang, Y., Sun, L., Yu, P. S., et al. Graph foundation models: Concepts, opportunities and challenges. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.
- McCallum, A. K., Nigam, K., Rennie, J., et al. Automating the construction of internet portals with machine learning. *Information Retrieval*, 3(2):127–163, 2000.
- Mernyei, P. and Cangea, C. Wiki-cs: A wikipedia-based benchmark for graph neural networks. *arXiv preprint arXiv:2007.02901*, 2020.
- Reimers, N. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019.
- Tan, Y., Lv, H., Huang, X., et al. Musegraph: Graph-oriented instruction tuning of large language models for generic graph mining. *arXiv preprint arXiv:2403.04780*, 2024a.
- Tan, Y., Zhou, Z., Lv, H., et al. Walklm: A uniform language model fine-tuning framework for attributed graph embedding. *Advances in Neural Information Processing Systems*, 36, 2024b.
- Tang, J., Yang, Y., Wei, W., et al. Graphgpt: Graph instruction tuning for large language models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 491–500, 2024.
- Touvron, H., Martin, L., Stone, K., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Wang, D., Zuo, Y., Li, F., et al. Llms as zero-shot graph learners: Alignment of gnn representations with llm token embeddings. *Advances in Neural Information Processing Systems*, 37:5950–5973, 2024a.
- Wang, D., Zuo, Y., Lu, G., et al. Unigte: Unified graph-text encoding for zero-shot generalization across graph tasks and domains. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Wang, H., Feng, S., He, T., et al. Can language models solve graph problems in natural language? *Advances in Neural Information Processing Systems*, 36, 2024b.
- Wang, J., Wu, J., Hou, Y., et al. Instructgraph: Boosting large language models via graph-centric instruction tuning and preference alignment. *arXiv preprint arXiv:2402.08785*, 2024c.
- Yan, H., Li, C., Long, R., et al. A comprehensive study on text-attributed graphs: Benchmarking and rethinking. *Advances in Neural Information Processing Systems*, 36: 17238–17264, 2023.
- Yang, Z., Cohen, W., and Salakhudinov, R. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, pp. 40–48. PMLR, 2016.
- Ye, R., Zhang, C., Wang, R., et al. Natural language is all a graph needs. *arXiv preprint arXiv:2308.07134*, 4(5):7, 2023.
- Zhang, G., Li, Z., Huang, J., et al. efraudcom: An e-commerce fraud detection system via competitive graph neural networks. *ACM Transactions on Information Systems (TOIS)*, 40(3):1–29, 2022.
- Zhang, M., Sun, M., Wang, P., et al. Graphtranslator: Aligning graph model to large language model for open-ended tasks. In *Proceedings of the ACM on Web Conference 2024*, pp. 1003–1014, 2024.
- Zhang, S., Dong, L., Li, X., et al. Instruction tuning for large language models: A survey. *ACM Computing Surveys*, 58(7):1–36, 2026.
- Zhang, Z., Wang, X., et al. Can large language models improve the adversarial robustness of graph neural networks? In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, 2025.
- Zhao, J., Zhuo, L., Shen, Y., et al. Graphtext: Graph reasoning in text space. *arXiv preprint arXiv:2310.01089*, 2023.

A. Related Work

In this section, we briefly discuss applications of LLMs to text-attributed graphs and existing benchmarks and evaluations of LLMs for graphs.

LLMs for Graphs. Existing applications of LLMs to TAGs can be broadly categorized into two lines of work. The first line, referred to as GTextLLMs, queries an LLM with textual representations of graphs. Early studies mainly focus on prompt design to help LLMs interpret textualized graph information, where a node is described by its textual features together with its neighborhood structure expressed in natural language (Chen et al., 2024b; Huang et al., 2023). Beyond directly listing neighboring information, subsequent works explore more structured textual representations, such as syntax trees (Zhao et al., 2023), random walks (Tan et al., 2024b), and code-like formats (Wang et al., 2024c). More recently, several studies (Wang et al., 2024c; Tan et al., 2024a) further fine-tune open-source LLMs (Touvron et al., 2023; Chiang et al., 2023) to improve their general capabilities on graph-related tasks. The second line of work is GTokenLLMs, which integrates real-world graphs into LLMs by encoding graph structures and textual features into token-level embeddings. As introduced in Section B, representative GTokenLLMs include InstructGLM (Ye et al., 2023), GraphGPT (Tang et al., 2024), GraphTranslator (Zhang et al., 2024), TEA-GLM (Wang et al., 2024a), and GOFA (Kong et al., 2025).

Benchmarks and Evaluations of LLMs for Graphs. Several survey studies (Chen et al., 2024b; Huang et al., 2023) highlight the importance of label space design and structural information in applying LLMs to graph tasks. In addition, existing benchmarks and evaluations examine LLM capabilities across a range of graph-related problems, including graph reasoning (Guo et al., 2023), adversarial robustness (Guo et al., 2024), graph theory problems (Wang et al., 2024b; Fatemi et al., 2024; Li et al., 2024), and graph pattern understanding (Dai et al., 2024). Despite these efforts, current evaluations mainly focus on LLMs operating on textualized graphs, while a systematic evaluation of GTokenLLMs remains lacking. As a result, the understanding of graph tokens in GTokenLLMs remains underexplored.

B. An Introduction of Representative GTokenLLMs and How They Fit into the Framework

In this section, we investigate six representative GTokenLLMs: LLaGA (Chen et al., 2024a), InstructGLM (Ye et al., 2023), GraphGPT (Tang et al., 2024), GraphTranslator (Zhang et al., 2024), TEA-GLM (Wang et al., 2024a), and GOFA (Kong et al., 2025). We show that these models fit into our proposed framework.

LLaGA (Chen et al., 2024a) is a general framework that transforms graph-structured data into structure-aware token sequences and maps them into the LLM’s token embedding space via a learned projector, enabling LLMs to perform graph tasks while preserving their general-purpose language capabilities. Specifically, in stage 1, LLaGA employs an off-the-shelf text encoder, such as Sentence-BERT (Reimers, 2019) or SimTeG (Duan et al., 2023), to obtain node embeddings from textual attributes, and transforms graph structures into structure-aware GE sequences using predefined templates. In stage 2, it projects GEs into GTs via a learned multilayer perceptron (MLP). Finally, these GTs are used as prefix tokens to query the LLM for node classification, link prediction, and node description tasks in stage 3. Notably, the LLM is kept frozen to preserve its comprehension capabilities and reduce training costs.

InstructGLM (Ye et al., 2023) projects shallow node embeddings (e.g., Bag-of-Words (Harris, 1954) features on Cora) into graph tokens and integrates them with structure-aware prompt templates, enabling instruction-tuned LLMs to perform graph-related tasks. Specifically, InstructGLM adopts shallow, task-specific graph features as GEs, which are mapped into GTs via a lightweight MLP projector to align with the LLM input space. The model further incorporates explicit, language-based descriptions of local graph structure (e.g., “<node 1> is connected with <node 2> within one hop”) in the prompt template and fine-tunes the LLM with LoRA (Hu et al., 2022) for node classification and link prediction tasks.

GraphGPT (Tang et al., 2024) aims to align LLMs’ reasoning ability with graph-domain structural knowledge learned by a pretrained GNN, thereby improving the generalization of graph learning. Specifically, GraphGPT employs a text-grounded GNN trained with CLIP-style contrastive alignment to obtain GEs, where node features are text embeddings encoded by BERT (Devlin et al., 2019). These GEs are then projected into GTs via a lightweight MLP and organized into prompt templates that represent a central node together with one of its neighbors when querying the LLM.

GraphTranslator (Zhang et al., 2024) aims to bridge pretrained graph models and LLMs by translating graph embeddings into query tokens, enabling LLMs to serve as an open-ended interface for graph learning. Specifically, GraphTranslator first adopts a pretrained GraphSAGE (Hamilton et al., 2017) to obtain GEs. It then employs a Translator module based on a

Table 7. Example of GTextLLM prompts.

Type	Instruction
Zero-Shot	Paper: {text feature} Task: Please predict the most appropriate category for the paper. Your answer should be chosen from {classes}.
1-Hop Title+Label	Input: Given the content and citation relationship of papers, determine the category of papers. Papers with citation relationships are more likely to share the same category. There are following categories: {classes} {textual feature of central node and its one-hop neighbors} Which category does Paper 1 belong to?

Q-former to align these GEs with textual representations of the target node and its neighborhood, producing a set of query tokens (i.e., GTs). These query tokens are used as prefix tokens to query a frozen LLM, enabling various open-ended graph tasks while preserving the original graph model for predefined tasks.

TEA-GLM (Wang et al., 2024a) aligns GNN representations learned via contrastive learning with the LLM’s input space, enabling cross-dataset and cross-task zero-shot graph learning. Specifically, TEA-GLM pre-trains a GNN using contrastive learning and introduces a PCA-based feature alignment in the LLM embedding space to obtain GEs, where node features are text embeddings from BERT (Devlin et al., 2019). It then trains a lightweight linear projector to map GEs into a fixed number of GTs without tuning the LLM. Finally, these GTs are injected into a unified prompt template to query the LLM for node classification and link prediction in zero-shot settings.

GOFA (Kong et al., 2025) proposes a generative graph language model by interleaving GNN layers with a frozen LLM compressor to compress graph structure and textual attributes into a fixed number of memory embeddings for downstream inference by another LLM. Specifically, GOFA interleaves GNN layers with transformer layers of an LLM compressor (Ge et al., 2024) to learn memory embeddings that capture both structural and attribute semantic information, whose final-layer outputs serve as GTs. These GTs are then directly used as query tokens for another LLM, enabling graph-conditioned generation and reasoning.

C. GTextLLM Prompts

We list the GTextLLM prompts used in Table 7, including two commonly used prompts from prior work (Chen et al., 2024b): “zero-shot”, which contains the textual description of the central node, and “1-hop”, which adds the text features of one-hop neighbors on top of the “zero-shot” prompt. In addition, we test four instruction types. Notably, the original instruction contains a textual description of the graph rather than graph tokens.

D. Experiment Details of GTEval

D.1. Dataset

We conduct experiments on seven widely used datasets from different domains: Cora (Yang et al., 2016), Cora-70 (McCallum et al., 2000), Pubmed (Yang et al., 2016), and OGBN-Arxiv (Hu et al., 2020) from citation networks; OGBN-Products (Hu et al., 2020) and Book-History (Yan et al., 2023) from an e-commerce network; and WikiCS (Mernyei & Cangea, 2020) from a web link network. Our data include shallow embeddings commonly used in classical methods, raw node texts, edge indices, node labels, label names, and masks for training, validation, and testing. All datasets are under the MIT License unless otherwise specified. Detailed descriptions of these datasets are provided below:

- **Cora** is a citation network of research papers in the computer science domain. In this dataset, each node represents a paper, and its raw text features include the title and abstract. An edge indicates a citation relationship between two papers. Each node label corresponds to the paper’s category.
- **Cora-70** is a more comprehensive version of the Cora dataset, containing more papers and more classes than the subset used in other experiments. This dataset, formally known as the “Cora Research Paper Classification Dataset”, provides a citation network for studying research paper classification in machine learning.
- **Pubmed** is a citation network of biomedical research papers. In this dataset, each node represents a paper and each edge denotes a citation relationship between two papers.
- **OGBN-Arxiv** is a citation network of papers and their citation relationships collected from the arXiv platform (Hu et al.,

Table 8. Dataset statistics and the LLM used by each model.

Model	LLM	Dataset	Domain	# Nodes	# Edges	# Classes	Train/Val/Test (%)
LLaGA	Vicuna-7B-v1.5	Cora	Citation	2,708	5,429	7	60.0/20.0/20.0
		Pubmed	Citation	19,717	44,338	3	60.0/20.0/20.0
		Arxiv	Citation	169,343	1,166,243	40	53.7/17.6/28.7
		Products	E-commerce	316,513	19,337,722	39	8.0/1.6/90.4
GraphGPT	Vicuna-7B-v1.5	Cora-70	Citation	25,120	182,280	70	40.7/13.7/13.6
		Pubmed	Citation	19,717	44,338	3	60.0/20.0/20.0
		Arxiv	Citation	169,343	1,166,243	40	53.7/17.6/28.7
InstructGLM	Llama-v1-7b	Cora	Citation	2,708	5,429	7	60.0/20.0/20.0
		Pubmed	Citation	19,717	44,338	3	60.0/20.0/20.0
		Arxiv	Citation	169,343	1,166,243	40	53.7/17.6/28.7
TEA-GLM	Vicuna-7B-v1.5	Cora-70	Citation	25,120	182,280	70	40.7/13.7/13.6
		Pubmed	Citation	19,717	44,338	3	60.0/20.0/20.0
		History	E-commerce	41,551	358,574	12	60.0/20.0/20.0
GOFA	Mistral-7B	Cora	Citation	2,708	5,429	7	5.2/18.5/76.4
		WikiCS	Web link	11,701	216,123	10	5.0/15.1/50.0
GraphTranslator	ChatGLM2-6B	Arxiv	Citation	169,343	1,166,243	40	53.7/17.6/28.7

2020). In this dataset, each node represents a paper and each edge denotes a citation relationship.

- **Book-History** is extracted from the Amazon-Books dataset and consists of items whose second-level label is “History”. In this dataset, nodes represent books and edges indicate that two books are frequently co-viewed. Each node label corresponds to the book’s third-level category. We use the book title and description as node text attributes. The task is to classify books into 24 and 12 categories, respectively.

- **OGBN-Products** is a co-purchase graph in which nodes represent product items on Amazon and edges indicate that two products are frequently co-purchased. This graph provides 47 categories.

- **WikiCS** is a web link network in which each node represents a Wikipedia page and each edge is a reference link between pages. For each node, the raw text includes the page title and content. Each node label corresponds to the page category.

D.2. Implementation Details

For all GTokenLLMs, we use their original code and the default hyper-parameter settings in the authors’ implementation. The sources are listed as follows:

- **LLaGA**: <https://github.com/VITA-Group/LLaGA>
- **InstructGLM**: <https://github.com/agiresearch/InstructGLM>
- **GraphGPT**: <https://github.com/HKUDS/GraphGPT>
- **TEA-GLM**: <https://github.com/W-rudder/TEA-GLM>
- **GOFA**: <https://github.com/JiaruiFeng/GOFA>
- **GraphTranslator**: <https://github.com/alibaba/GraphTranslator>

D.3. Computing Environment and Resources

We provide the source code for our test examples in the supplementary material, and the complete source code will be made publicly available after the review. Due to file size limitations, the checkpoint of LLMs and datasets will be publicly available after the review. The implementation of the proposed GTEval utilized the PyG module. The experiments are conducted in a computing environment with the following specifications:

- **OS**: Linux ubuntu 5.15.0-102-generic.
- **CPU**: Intel(R) Xeon(R) Platinum 8358 CPU @ 2.60GHz.
- **GPU**: NVIDIA A800 80GB.